I used the The first thing i did was give Claude and gemini the rubric and instructions along with the prompt “generate the code i need”. I decided to use the claude code because it was shorter. I removed the comments and redid them myself.

Claude gave me this raw code:

```
/*
 * Program Name : EECS 348 Assignment 4 - Recursive Object-Oriented Sudoku Solver
 * Description  : Reads Sudoku puzzles from puzzle1.txt ... puzzle5.txt, solves each one
 *                with a recursive depth-first search with backtracking, and prints the
 *                original puzzle followed by EVERY solution found (or "No solution found").
 * Inputs       : puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, puzzle5.txt
 *                (9 rows of 9 characters; digits 1-9 are givens, '_' is a blank cell)
 * Output       : For each file: the file name, the puzzle, and all solutions
 *                (or "No solution found") printed to the console.
 * Collaborators: [LIST ANY PEOPLE YOU WORKED WITH, OR WRITE "None"]
 * Other sources: Initial code generated by Claude (Anthropic GenAI), accessed through the
 *                Claude web chat at claude.ai. [ADD THE OTHER GenAI AND ANY URLs YOU
USED]
 *                Modifications and comments by the author.
 * Author       : [YOUR FULL NAME]
 * Creation date: [DATE YOU CREATED THIS FILE]
 * Revision date: [DATE OF LAST REVISION]
 */

#include <iostream>   // console output (std::cout)
#include <fstream>    // reading puzzle files (std::ifstream)
#include <string>     // std::string for file names
#include <vector>     // std::vector to store the solutions
#include <array>      // std::array for the fixed 9x9 grid

// ---------------------------------------------------------------------------
// Class Puzzle: owns a 9x9 grid and knows how to load and print itself.
// ---------------------------------------------------------------------------
class Puzzle {
public:
    static const int SIZE = 9;                // grid is SIZE x SIZE (9 x 9)

    Puzzle() {                                // default constructor
        for (auto& row : cells) row.fill(0);   // start with every cell empty (0 = blank)
    }

    // Reads a puzzle from a file; returns false if the file is missing or malformed.
```

```
    bool loadFromFile(const std::string& fileName) {
        std::ifstream in(fileName);            // open the file for reading
        if (!in) return false;                 // could not open -> report failure
        int count = 0;                         // how many cells we have read so far
        char ch;                               // one character at a time
        while (count < SIZE * SIZE && in.get(ch)) {  // stop after 81 cells or end of file
            if (ch >= '1' && ch <= '9') {              // a given digit
                cells[count / SIZE][count % SIZE] = ch - '0';  // convert char to int, store it
                count++;                               // move to next cell
            } else if (ch == '_' || ch == '0' || ch == '.') {  // a blank square
                cells[count / SIZE][count % SIZE] = 0;         // store 0 for blank
                count++;                                       // move to next cell
            }                                      // anything else (spaces, newlines) is skipped
        }
        return count == SIZE * SIZE;           // success only if we got all 81 cells
    }

    int get(int r, int c) const { return cells[r][c]; }     // read one cell
    void set(int r, int c, int v) { cells[r][c] = v; }      // write one cell

    // Prints the grid, one row per line, '_' for blanks, with gaps between 3x3 blocks.
    void print(std::ostream& out) const {
        for (int r = 0; r < SIZE; r++) {                    // go through each row
            if (r > 0 && r % 3 == 0) out << "------+-------+------\n";  // horizontal block divider
            for (int c = 0; c < SIZE; c++) {                // go through each column
                if (c > 0 && c % 3 == 0) out << "| ";       // vertical block divider
                if (cells[r][c] == 0) out << "_";           // blank cell
                else out << cells[r][c];                    // filled cell
                out << " ";                                 // space between cells
            }
            out << "\n";                                    // end of row
        }
    }

private:
    std::array<std::array<int, SIZE>, SIZE> cells;          // the grid itself
};

// ---------------------------------------------------------------------
// Class Solver: finds ALL solutions of a Puzzle using recursive backtracking.
// Bitmasks track which digits are already used in each row, column, and block,
// so checking a candidate is a couple of bit operations instead of a grid scan.
// ---------------------------------------------------------------------
class Solver {
```

```
public:
    explicit Solver(const Puzzle& start) : board(start) {   // copy the puzzle to work on
        for (int i = 0; i < 9; i++) rowUsed[i] = colUsed[i] = boxUsed[i] = 0;  // nothing used yet
    }

    // Public entry point: sets up the masks, runs the search, returns solution count.
    int solve() {
        solutions.clear();                      // forget any earlier results
        if (!initMasks()) return 0;             // givens already break the rules -> no solution
        search(countGivens());                  // start recursion with the givens already filled
        return (int)solutions.size();           // how many solutions were found
    }

    const std::vector<Puzzle>& getSolutions() const { return solutions; }  // access results

private:
    Puzzle board;                               // working copy of the puzzle
    int rowUsed[9], colUsed[9], boxUsed[9];     // bit d-1 set => digit d is used in that unit
    std::vector<Puzzle> solutions;              // every complete solution discovered

    static int boxIndex(int r, int c) { return (r / 3) * 3 + (c / 3); }  // which 3x3 block a cell is in

    int countGivens() const {                   // count the cells that start out filled
        int n = 0;
        for (int r = 0; r < 9; r++)
            for (int c = 0; c < 9; c++)
                if (board.get(r, c) != 0) n++;
        return n;
    }

    // Fills the masks from the given digits; returns false if a given digit repeats in a unit.
    bool initMasks() {
        for (int r = 0; r < 9; r++) {
            for (int c = 0; c < 9; c++) {
                int d = board.get(r, c);        // digit in this cell (0 if blank)
                if (d == 0) continue;           // blanks add nothing
                int bit = 1 << (d - 1);         // mask bit for this digit
                int b = boxIndex(r, c);         // block of this cell
                if ((rowUsed[r] & bit) || (colUsed[c] & bit) || (boxUsed[b] & bit))
                    return false;               // duplicate found -> puzzle is invalid
                rowUsed[r] |= bit;              // mark digit used in row
                colUsed[c] |= bit;              // mark digit used in column
                boxUsed[b] |= bit;              // mark digit used in block
            }
```

```
        }
        return true;                            // all givens are consistent
    }

    // Recursive depth-first search. 'filled' = number of cells filled so far.
    void search(int filled) {
        if (filled == 81) {                     // base case: every cell is filled
            solutions.push_back(board);         // save a copy of this solution
            return;                             // go back to look for more
        }

        // Pick the empty cell with the FEWEST candidates (fail-first heuristic).
        int bestR = -1, bestC = -1, bestMask = 0, bestCount = 10;
        for (int r = 0; r < 9; r++) {
            for (int c = 0; c < 9; c++) {
                if (board.get(r, c) != 0) continue;             // skip filled cells
                int mask = ~(rowUsed[r] | colUsed[c] | boxUsed[boxIndex(r, c)]) & 0x1FF;  // allowed
digits
                int cnt = __builtin_popcount(mask);             // how many digits are allowed
                if (cnt < bestCount) {                          // better (more constrained) cell?
                    bestCount = cnt; bestMask = mask;           // remember its candidate info
                    bestR = r; bestC = c;                       // remember its position
                }
            }
        }
        if (bestCount == 0) return;             // dead end: an empty cell has no candidates,
backtrack

        int b = boxIndex(bestR, bestC);         // block of the chosen cell
        for (int d = 1; d <= 9; d++) {          // try each digit 1..9
            int bit = 1 << (d - 1);             // mask bit for this digit
            if (!(bestMask & bit)) continue;    // skip digits that are not candidates
            board.set(bestR, bestC, d);         // place the digit
            rowUsed[bestR] |= bit; colUsed[bestC] |= bit; boxUsed[b] |= bit;  // mark it used
            search(filled + 1);                 // recurse with one more cell filled
            board.set(bestR, bestC, 0);         // undo the placement (backtrack)
            rowUsed[bestR] &= ~bit; colUsed[bestC] &= ~bit; boxUsed[b] &= ~bit;  // unmark it
        }
    }
};

// ---------------------------------------------------------------------------
// Class PuzzleRunner: loads one file, runs the solver, and reports the results.
// ---------------------------------------------------------------------------
```

```
class PuzzleRunner {
public:
    explicit PuzzleRunner(const std::string& file) : fileName(file) {}  // remember the file name

    void run() const {
        std::cout << "==============================\n";
        std::cout << fileName << "\n";                    // print the puzzle file name
        Puzzle puzzle;                                    // empty puzzle to fill from file
        if (!puzzle.loadFromFile(fileName)) {             // try to read the file
            std::cout << "Could not read a valid puzzle from this file.\n\n";
            return;                                       // nothing more we can do
        }
        std::cout << "Puzzle:\n";
        puzzle.print(std::cout);                          // show the starting puzzle

        Solver solver(puzzle);                            // build a solver for this puzzle
        int count = solver.solve();                       // run the recursive search
        if (count == 0) {                                 // nothing found
            std::cout << "\nNo solution found\n\n";
            return;
        }
        std::cout << "\nSolutions found: " << count << "\n"; // report how many
        int n = 1;                                        // solution counter for labels
        for (const Puzzle& s : solver.getSolutions()) {   // print every solution
            std::cout << "\nSolution " << n++ << ":\n";
            s.print(std::cout);
        }
        std::cout << "\n";
    }

private:
    std::string fileName;                                 // name of the puzzle file to process
};

int main() {
    for (int i = 1; i <= 5; i++) {                        // puzzle1.txt through puzzle5.txt
        PuzzleRunner runner("puzzle" + std::to_string(i) + ".txt");  // build a runner for this file
        runner.run();                                     // load, solve, print
    }
    return 0;                                             // normal exit
}
```

Gemini's raw code:

```
/**
 * @file Assignment4.cpp
 * @brief EECS 348 Assignment 4 - Recursive Object-Oriented Sudoku Solver
 * @details Reads five Sudoku puzzle files (puzzle1.txt through puzzle5.txt)
 *          containing numbers and underscores for blanks, and solves them
 *          using a recursive backtracking algorithm implemented via C++ classes.
 * @inputs puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, puzzle5.txt
 * @outputs Prints the puzzle filename, initial board, and solved board(s) to the console.
 * @collaborators None
 * @sources EECS 348 Assignment 4 Instructions, Standard C++ Documentation
 * @author Nathaniel Lanter
 * @date 2026-10-16
 */

#include <iostream>
#include <fstream>
#include <vector>
#include <string>

// Class representing an individual Sudoku Cell
class SudokuCell {
private:
    int value; // Stored digit 1-9, or 0 if empty

public:
    // Default constructor initializes cell to empty (0)
    SudokuCell() {
        value = 0; // Authored by Nathaniel Lanter
    }

    // Parameterized constructor to set cell value
    SudokuCell(int val) {
        value = val; // Authored by Nathaniel Lanter
    }

    // Get the current value of the cell
    int getValue() const {
        return value; // Authored by Nathaniel Lanter
    }

    // Set the value of the cell
    void setValue(int val) {
        value = val; // Authored by Nathaniel Lanter
```

```
    }

    // Check if the cell is currently empty
    bool isEmpty() const {
        return value == 0; // Authored by Nathaniel Lanter
    }
};

// Class managing the 9x9 Sudoku board, validation, and recursive solving logic
class SudokuBoard {
private:
    SudokuCell board[9][9]; // 9x9 grid of SudokuCell objects

    // Helper method to check if placing a number is valid according to Sudoku rules
    bool isValid(int row, int col, int num) const {
        // Loop through each column in the current row to check for duplicates
        for (int c = 0; c < 9; ++c) {
            // Check if the number already exists in the same row (excluding current cell)
            if (board[row][c].getValue() == num) {
                return false; // Duplicate found in row
            }
        }

        // Loop through each row in the current column to check for duplicates
        for (int r = 0; r < 9; ++r) {
            // Check if the number already exists in the same column (excluding current cell)
            if (board[r][col].getValue() == num) {
                return false; // Duplicate found in column
            }
        }

        // Determine the top-left starting row index for the 3x3 sub-grid
        int startRow = row - (row % 3);
        // Determine the top-left starting column index for the 3x3 sub-grid
        int startCol = col - (col % 3);

        // Loop through the 3x3 sub-grid rows
        for (int r = 0; r < 3; ++r) {
            // Loop through the 3x3 sub-grid columns
            for (int c = 0; c < 3; ++c) {
                // Check if the number already exists within the 3x3 block
                if (board[startRow + r][startCol + c].getValue() == num) {
                    return false; // Duplicate found in 3x3 sub-grid
                }
```

```
            }
        }

        return true; // Placement is valid
    }

    // Recursive backtracking helper method to solve the puzzle
    bool solveRecursive(int row, int col) {
        // Base case: if we have moved past the last row (row 8), the puzzle is fully solved
        if (row == 9) {
            return true; // Solution found successfully
        }

        // Move to the next row if we have reached the end of the current column
        int nextRow = (col == 8) ? row + 1 : row;
        // Move to the next column or wrap around to the first column (0)
        int nextCol = (col == 8) ? 0 : col + 1;

        // If the current cell is already filled, skip to the next cell recursively
        if (!board[row][col].isEmpty()) {
            return solveRecursive(nextRow, nextCol); // Proceed without modifying current cell
        }

        // Try digits 1 through 9 sequentially in the empty cell
        for (int num = 1; num <= 9; ++num) {
            // Check if placing the current digit obeys Sudoku constraints
            if (isValid(row, col, num)) {
                // Temporarily place the number into the cell
                board[row][col].setValue(num);

                // Recursively attempt to solve the rest of the board with this choice
                if (solveRecursive(nextRow, nextCol)) {
                    return true; // Propagate success upward
                }

                // If placement leads to a dead end, backtrack by resetting the cell to empty
                board[row][col].setValue(0); // Reset cell value
            }
        }

        return false; // Trigger backtracking when no candidates work
    }

public:
```

```
// Default constructor initializes an empty board
SudokuBoard() {
    // Grid elements initialized via SudokuCell default constructor
} // Authored by Nathaniel Lanter

// Load board state from a specified text file containing numbers and underscores
bool loadFromFile(const std::string& filename) {
    // Open the puzzle file stream
    std::ifstream file(filename);
    // Check if the file opened successfully
    if (!file.is_open()) {
        std::cerr << "Error: Could not open file " << filename << std::endl;
        return false; // File open failed
    }

    // Loop through each row of the 9x9 grid
    for (int r = 0; r < 9; ++r) {
        // Loop through each column of the current row
        for (int c = 0; c < 9; ++c) {
            char ch;
            // Read a character from the file
            if (!(file >> ch)) {
                std::cerr << "Error: Format error in file " << filename << std::endl;
                return false; // Malformed file format
            }

            // Check if the character represents an empty space (underscore)
            if (ch == '_') {
                board[r][c] = SudokuCell(0); // Set cell as empty
            } else if (ch >= '1' && ch <= '9') {
                board[r][c] = SudokuCell(ch - '0'); // Convert char digit to integer and store
            } else {
                std::cerr << "Error: Invalid character '" << ch << "' in file " << filename << std::endl;
                return false; // Invalid character found
            }
        }
    }

    file.close(); // Close the file stream
    return true;  // Successfully loaded board
}

// Print the current state of the Sudoku board to standard output
void display() const {
```

```
        // Loop through each row for printing
        for (int r = 0; r < 9; ++r) {
            // Print horizontal separators for 3x3 block visual clarity
            if (r > 0 && r % 3 == 0) {
                std::cout << "---------------------\n";
            }

            // Loop through each column in the row
            for (int c = 0; c < 9; ++c) {
                // Print vertical separators for 3x3 block visual clarity
                if (c > 0 && c % 3 == 0) {
                    std::cout << "| ";
                }

                // Retrieve the cell value
                int val = board[r][c].getValue();
                // Print underscore if empty, or the digit if filled
                if (val == 0) {
                    std::cout << "_ ";
                } else {
                    std::cout << val << " ";
                }
            }
            std::cout << std::endl; // End line for the current row
        }
    } // Authored by Nathaniel Lanter

    // Public wrapper method to execute the recursive backtracking solver
    bool solve() {
        return solveRecursive(0, 0); // Initiate recursion from top-left cell (0,0)
    } // Authored by Nathaniel Lanter
};

// Main driver function to process all required puzzle files
int main() {
    // Array containing the names of the five required puzzle files
    std::string puzzleFiles[5] = {
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
    };
```

```
    // Loop through each puzzle file name in the array
    for (int i = 0; i < 5; ++i) {
        // Instantiate a new SudokuBoard object for the current puzzle
        SudokuBoard puzzleSolver;

        // Print header indicating the current puzzle file being processed
        std::cout << "================================\n";
        std::cout << "Puzzle File: " << puzzleFiles[i] << "\n";
        std::cout << "================================\n";

        // Attempt to load the puzzle from the corresponding text file
        if (puzzleSolver.loadFromFile(puzzleFiles[i])) {
            // Print the initial unsolved puzzle board configuration
            std::cout << "Initial Puzzle:\n";
            puzzleSolver.display();
            std::cout << "\n------------------------------\n";

            // Execute the recursive solver and check if a solution exists
            if (puzzleSolver.solve()) {
                // Print the successfully solved puzzle board
                std::cout << "Solution Found:\n";
                puzzleSolver.display();
            } else {
                // Print notification message if no valid solution exists
                std::cout << "No solution found.\n";
            }
        } else {
            // Print notification if loading the puzzle file failed
            std::cout << "Failed to load " << puzzleFiles[i] << std::endl;
        }

        std::cout << "\n\n"; // Separate outputs between different puzzle files
    }

    return 0; // Return success status code to operating system
} // Authored by Nathaniel Lanter
```